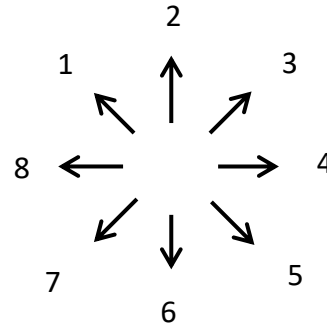


[RVHS/2025/Q2 Cont']

Task 2.1

Implement the function `valid_dir(board, curr_pos, direction)` to check whether there is an adjacent tile in the given direction from the current tile position `curr_pos`. If there is no tile in the given direction indicated by `direction`, return `False`. Otherwise, return `True`.

0	A	E	T	S
1	T	O	R	Y
2	I	S	H	E
3	O	M	E	A
	0	1	2	3



For example,

```
>>> board =  
[["A","E","T","S"],  
 ["T","O","R","Y"],  
 ["I","S","H","E"],  
 ["O","M","E","A"]]  
>>> valid_dir(board, (0, 0) , 1)  
False  
>>> valid_dir(board, (0, 0) , 2)  
False  
>>> valid_dir(board, (0, 0) , 3)  
False  
>>> valid_dir(board, (0, 0) , 4)  
True  
>>> valid_dir(board, (0, 0) , 5)  
True  
>>> valid_dir(board, (0, 0) , 6)  
True  
>>> valid_dir(board, (0, 0) , 7)  
False  
>>> valid_dir(board, (0, 0) , 8)  
False
```

The result above occurs because the tile at position `(0, 0)` has only three adjacent tiles — in directions 4, 5, and 6. Hence they return `True`.

For your own testing, you can use the code provided for **Task 2** in `“use_code.txt”` to initialize board.

[RVHS/2025/Q2 Cont']

Task 2.2

Implement the function `result_word(board, move)` to return the English word identified by `move`. Use `valid_dir()` in **Task 2.1** to check if there is any invalid direction in `move`. If `move` cannot successfully identify an English word, return `None`. Otherwise return the English word in string.

For example,

```
>>> result_word(board, [2,1,7,4,4])
SOME
>>> result_word(board, [2,2,4,1,5])
HERE
>>> result_word(board, [2,2,4,1,1])
HERE
>>> result_word(board, [2,2,4,1,1,1])
None
```

Test your function using `valid_moves` and `invalid_moves` that are provided for **Task 2** in the file `"useful_code.txt"`.

[6]

Task 2.3

Each word that is identified by a move successfully is given a score. The score is calculated by adding the number that is assigned to the tile each time the tile is visited in a move.

The number assigned to each tile in the 4 x 4 board is represented by a 2D array as follows.

```
score_board = [[1,2,3,1],[2,2,3,1],[1,2,2,1],[1,4,2,1]]
```


Using the score board above and the move `[2,1,4,1,4,2]`. The score for the word "SHORT" is $2 + 2 + 2 + 3 + 3 = 12$.

Implement the function `result_score(score_board, move)` to return the score of the word identified by `move`. You can assume that `move` is always a valid move.

Test your function using `score_board` and `valid_moves` that are provided for **Task 2** in the file `"useful_code.txt"`.

[4]

[RVHS/2025/Q2 Cont']

Task 2.4

Implement the function `word_scores(board, score_board, moves)` which returns a list of lists that contains all the words that can be identified by `moves` and their respective scores.

You can assume that all the move in `moves` are valid.

For example,

```
>>> board =
[[ "A", "E", "T", "S"],
 [ "T", "O", "R", "Y"],
 [ "I", "S", "H", "E"],
 [ "O", "M", "E", "A"]]
>>> score_board =
[[1,2,3,1],
 [2,2,3,1],
 [1,2,2,1],
 [1,4,2,1]]
>>> valid_moves =
[[2,1,4,1,4,2],
 [2,1,7,4,4],
 [2,1,1,4,4,5],
 [0,3,8,7,4,4],
 [1,2,8,6,5],
 [2,0,4]]
>>> word_scores(board, score_board, valid_moves)
[['SHORT', 12], ['SOME', 9], ['STORE', 10], ['STORY', 10],
 ['ROSE', 9], ['IS', 3]]
```

Test your function using `board`, `score_board` and `valid_moves` that are provided for **Task 2** in the file *“useful_code.txt”*.

[3]

[RVHS/2025/Q2 Cont']

Task 2.5

Implement the function `display_word_score_sorted(board, score_board, valid_moves)` that does the following:

- Use the function in **Task 2.4** to receive a list of words and their respective scores.
- Perform bubble sort on the list based on their scores in descending order. For those words that have the same score, sort them in alphabetical order.
- Display them using the following format.

WORD	SCORE
SHORT	12
SHEER	10
SHORE	10
STORE	10
STORY	10
MOST	9
SOME	9
STATE	9
ROT	8
SHOE	8
SHOT	8
HERE	7
TOE	7
ME	6
SHE	6
TEA	6
ATE	5
EAT	5
OR	5
AT	3
HE	3
IS	3

Test your function using `board`, `score_board` and `more_moves` that are provided for **Task 2** in the file `"useful_code.txt"`.

[8]